

Data Cleaning & Feature Engineering

Practical Session 2 — Lab Sheet

B.Tech (Computer Engineering) — III Year
Machine Learning Laboratory

June 29, 2026

Instructions for Students: Work through each topic in order. Read the *Concept* box, run the *Example Code* yourself in a Jupyter Notebook / Google Colab / VS Code, and then solve the *Exercise* underneath *before* moving to the next topic. Keep all your code in a single notebook named `lab2_yourrollno.ipynb`. This sheet builds directly on Practical Session 1 — keep that notebook handy for reference.

Contents

From Clean Toy Data to Real-World Mess	2
Setup: Generating a Realistic Messy Dataset	2
1 Part I — Data Cleaning	4
1.1 Diagnosing Data Quality Issues	4
1.2 Removing Duplicate Records	4
1.3 Fixing Data Types	5
1.4 Imputing Missing Values	5
1.5 Detecting Outliers	6
1.6 Treating Outliers	7
1.7 Cleaning Messy Text & Categorical Values	7
2 Part II — Feature Engineering	9
2.1 Feature Scaling	9
2.2 Encoding Categorical Variables (scikit-learn)	10
2.3 Binning Continuous Variables	11
2.4 Extracting Features from Dates	11
2.5 Transforming Skewed Features	12
2.6 Interaction & Polynomial Features	12
2.7 Quick Feature Selection	13
3 Capstone Exercise — Putting It All Together	15
Appendix: Quick Reference Cheat Sheet	16

From Clean Toy Data to Real-World Mess

In Session 1 you worked with the Iris and Wine datasets exactly as scikit-learn provides them: no missing values, no duplicates, no typos, correct data types throughout. **Real datasets are never this polite.** Sensors fail and leave blanks, the same record gets logged twice, someone types "Pune " in one row and "PUNE" in another, and a numeric column quietly turns into text because one entry says "unknown". **Data Cleaning** is the process of detecting and fixing these problems. **Feature Engineering** is the process of reshaping clean data into the form a model actually learns best from – scaled, encoded, binned, and enriched with new derived columns. Industry surveys consistently report that data scientists spend more time on these two activities than on modeling itself, which is exactly why they get a full session before any model is trained.

How this sheet uses data. We will deliberately inject realistic problems into the same Iris and Wine datasets from Session 1 – missing values, duplicate rows, outliers, inconsistent text categories, and wrong data types – using two small helper functions defined below. As before, **Iris powers every example**, and **Wine powers every exercise**, so you must adapt each technique rather than copy it.

Setup: Generating a Realistic Messy Dataset

Concept

Copy both functions below to the top of your notebook *once*. Each one takes a fixed `seed` so the “mess” is reproducible – you and your classmates (and your instructor) will see the exact same missing values, duplicates, and outliers every time it is called. Every topic in Part I below calls one of these functions fresh, so topics are independent of each other.

Example Code

```
import numpy as np
import pandas as pd
from sklearn.datasets import load_iris, load_wine

def make_dirty_iris(seed=0):
    iris = load_iris()
    df = pd.DataFrame(iris.data,
                      columns=['sepal_length', 'sepal_width', 'petal_length', 'petal_width'])
    df['target'] = iris.target
    rng = np.random.default_rng(seed)

    # a messy categorical column: same 3 sites, written inconsistently
    sites = np.array(['North', 'South', 'East'])
    region = sites[rng.integers(0, 3, size=len(df))]
    variants = {'North': ['North', ' north', 'NORTH', 'Nort'],
                'South': ['South', 'south ', 'SOUTH', 'South'],
                'East': ['East', ' east', 'EAST', 'Esat']}
    df['region'] = [rng.choice(variants[r]) for r in region]

    # a date column stored as plain text
    base = pd.Timestamp('2023-01-01')
    offsets = rng.integers(0, 300, size=len(df))
    df['collection_date'] = [str((base + pd.Timedelta(days=int(o))).date())
                             for o in offsets]

    # missing values in two numeric columns
```

```

for col in ['sepal_length', 'sepal_width']:
    miss_idx = rng.choice(df.index, size=6, replace=False)
    df.loc[miss_idx, col] = np.nan

# a few extreme outliers
out_idx = rng.choice(df.index, size=3, replace=False)
df.loc[out_idx, 'sepal_length'] = df.loc[out_idx, 'sepal_length'] * 6

# a numeric column corrupted into text (stray spaces + 'unknown')
pl = df['petal_length'].astype(str)
junk_idx = rng.choice(df.index, size=4, replace=False)
pl.loc[junk_idx] = 'unknown'
space_idx = rng.choice(df.index, size=4, replace=False)
pl.loc[space_idx] = pl.loc[space_idx].apply(lambda v: f' {v} ')
df['petal_length'] = pl

# duplicate rows (added LAST so the copies are exact duplicates)
dup_idx = rng.choice(df.index, size=5, replace=False)
df = pd.concat([df, df.loc[dup_idx]], ignore_index=True)

return df.sample(frac=1, random_state=seed).reset_index(drop=True)

def make_dirty_wine(seed=1):
    wine = load_wine()
    df = pd.DataFrame(wine.data, columns=wine.feature_names)
    df['target'] = wine.target
    rng = np.random.default_rng(seed)

    regions = np.array(['Tuscany', 'Piedmont', 'Veneto'])
    region = regions[rng.integers(0, 3, size=len(df))]
    variants = {'Tuscany': ['Tuscany', ' tuscany', 'TUSCANY', 'Toscany'],
                'Piedmont': ['Piedmont', 'piedmont ', 'PIEDMONT', 'Piedmont'],
                'Veneto': ['Veneto', ' veneto', 'VENETO', 'Veneto']}
    df['storage_region'] = [rng.choice(variants[r]) for r in region]

    base = pd.Timestamp('2022-09-01')
    offsets = rng.integers(0, 120, size=len(df))
    df['bottling_date'] = [str((base + pd.Timedelta(days=int(o))).date())
                           for o in offsets]

    for col in ['magnesium', 'proline']:
        miss_idx = rng.choice(df.index, size=7, replace=False)
        df.loc[miss_idx, col] = np.nan

    out_idx = rng.choice(df.index, size=4, replace=False)
    df.loc[out_idx, 'color_intensity'] = df.loc[out_idx, 'color_intensity'] * 5

    av = df['ash'].astype(str)
    junk_idx = rng.choice(df.index, size=5, replace=False)
    av.loc[junk_idx] = 'NA'
    space_idx = rng.choice(df.index, size=5, replace=False)
    av.loc[space_idx] = av.loc[space_idx].apply(lambda v: f' {v} ')
    df['ash'] = av

    dup_idx = rng.choice(df.index, size=6, replace=False)
    df = pd.concat([df, df.loc[dup_idx]], ignore_index=True)

```

```
return df.sample(frac=1, random_state=seed).reset_index(drop=True)
```

1 Part I — Data Cleaning

1.1 Diagnosing Data Quality Issues

Concept

Before fixing anything, find out what is actually wrong. `.info()` reveals column dtypes and non-null counts at a glance, `.isnull().sum()` counts missing values per column, `.duplicated().sum()` counts exact duplicate rows, and `.describe()` often exposes outliers through suspicious min/max values. Make this four-command check a reflex every time you load a new dataset.

Example Code

```
df = make_dirty_iris()

print(df.info()) # dtypes + non-null counts
print(df.isnull().sum()) # missing values per column
print(df.duplicated().sum()) # number of exact duplicate rows
print(df.describe()) # watch the min/max rows for outliers
```

Exercise

1. Run `make_dirty_wine()` and the four diagnostic commands above on it. Report, in a comment, how many missing values, duplicates, and suspiciously-typed columns you find.
2. Looking at `.describe()`, which numeric column has an unbelievable maximum value? By how many times does it exceed the next highest value?
3. Print `df['storage_region'].unique()` and count how many *distinct spellings* exist for what should really be just 3 categories.

1.2 Removing Duplicate Records

Concept

Duplicate rows silently bias a model – a repeated sample gets extra weight during training and can leak between train and test splits. `.duplicated()` flags repeated rows (keeping the first occurrence as not-duplicate), and `.drop_duplicates()` removes them. Use the `subset` parameter when you only want to treat specific columns as defining a “duplicate” (e.g. ignoring an auto-generated ID column).

Example Code

```
df = make_dirty_iris()

print(df.duplicated().sum()) # how many duplicate rows?
clean = df.drop_duplicates() # drop exact duplicate rows
print(clean.shape)

# only compare a subset of columns when deciding what counts as duplicate
clean2 = df.drop_duplicates(subset=['sepal_length', 'sepal_width', 'petal_width'])
print(clean2.shape)
```

Exercise

1. On `make_dirty_wine()`, count and then remove exact duplicate rows. Report the shape before and after.
2. Use `keep='last'` instead of the default `keep='first'` when dropping duplicates. Does the resulting shape change? Explain why or why not in a one-line comment.
3. Drop duplicates considering only the `alcohol` and `color_intensity` columns. Is the result the same size as dropping *full-row* duplicates? Why might it differ?

1.3 Fixing Data Types

Concept

A column that *looks* numeric but is stored as text (often because of stray whitespace or a placeholder like "unknown") will silently break any arithmetic or model training. `.str.strip()` removes whitespace, `pd.to_numeric(..., errors='coerce')` converts text to numbers and turns anything unconvertible into NaN, `pd.to_datetime()` parses date strings, and `.astype('category')` marks a column as categorical (saving memory and enabling category-aware operations).

Example Code

```
df = make_dirty_iris()

print(df['petal_length'].dtype) # currently text
cleaned = df['petal_length'].str.strip() # remove stray spaces
cleaned = pd.to_numeric(cleaned, errors='coerce') # 'unknown' → NaN
df['petal_length'] = cleaned
print(df['petal_length'].dtype, df['petal_length'].isnull().sum())

df['collection_date'] = pd.to_datetime(df['collection_date'])
print(df['collection_date'].dtype)

df['target'] = df['target'].astype('category')
print(df['target'].dtype)
```

Exercise

1. On `make_dirty_wine()`, clean the `ash` column the same way: strip whitespace, then convert to numeric with `errors='coerce'`. How many values became NaN?
2. Convert `bottling_date` to a proper datetime column and print its new dtype.
3. Convert the `target` column to `category` dtype and use `.cat.categories` to print the category labels.

1.4 Imputing Missing Values

Concept

Session 1 introduced `fillna()` with a single constant. In practice you usually impute with a **statistic**: the mean or median for numeric columns (median is safer when outliers are present), the mode for categorical columns, or even a **group-wise** statistic (e.g. fill a missing measurement using the median *for that sample's class*, via `groupby().transform()`). scikit-learn's `SimpleImputer` does the same job inside an ML pipeline.

Example Code

```
df = make_dirty_iris()

df['sepal_width'] = df['sepal_width'].fillna(df['sepal_width'].median())

# group-wise imputation: fill using the median FOR THAT species (target)
df['sepal_length'] = df.groupby('target')['sepal_length'].transform(
    lambda s: s.fillna(s.median())
)
print(df[['sepal_length', 'sepal_width']].isnull().sum())

from sklearn.impute import SimpleImputer
imputer = SimpleImputer(strategy='mean')
df[['petal_width']] = imputer.fit_transform(df[['petal_width']])
print(df['petal_width'].isnull().sum())
```

Exercise

1. On `make_dirty_wine()`, impute magnesium with its median and proline with its mean. Confirm zero missing values remain in both.
2. Instead, impute magnesium **group-wise** using the median *per target class*. Does the imputed value differ much from the overall (non-grouped) median? Print both to compare.
3. Use `sklearn.impute.SimpleImputer` with `strategy='most_frequent'` to impute a categorical column of your choice (you will need to introduce a missing value into `storage_region` first, e.g. via `df.loc[0, 'storage_region'] = np.nan`).

1.5 Detecting Outliers**Concept**

An outlier is a value far enough from the rest of the data that it may distort statistics or model training. The **IQR method** flags anything below $Q1 - 1.5 \times IQR$ or above $Q3 + 1.5 \times IQR$, where $IQR = Q3 - Q1$. The **Z-score method** flags anything more than (typically) 3 standard deviations from the mean. IQR is more robust when the data itself contains extreme values, since it does not depend on the mean/standard deviation being reliable in the first place.

Example Code

```
df = make_dirty_iris()
df['sepal_length'] = df['sepal_length'].fillna(df['sepal_length'].median())

# --- IQR method ---
Q1 = df['sepal_length'].quantile(0.25)
Q3 = df['sepal_length'].quantile(0.75)
IQR = Q3 - Q1
lower, upper = Q1 - 1.5 * IQR, Q3 + 1.5 * IQR
outliers_iqr = df[(df['sepal_length'] < lower) | (df['sepal_length'] > upper)]
print(lower, upper, outliers_iqr.shape[0])

# --- Z-score method ---
z = (df['sepal_length'] - df['sepal_length'].mean()) / df['sepal_length'].std()
outliers_z = df[z.abs() > 3]
print(outliers_z.shape[0])
```

Exercise

1. On `make_dirty_wine()`, after imputing missing values, use the IQR method to detect outliers in `color_intensity`. How many rows are flagged?
2. Detect outliers in the same column using the Z-score method with a threshold of 3. Do you get the same rows as the IQR method?
3. Try a stricter Z-score threshold of 2.5. Does the number of flagged outliers increase or decrease? Explain why in one sentence.

1.6 Treating Outliers

Concept

Once detected, you generally have two choices: **remove** the offending rows (only safe when you are confident they are data-entry errors, and you can afford to lose the rows), or **cap** them at the IQR boundary – also called **winsorizing** – which keeps the row but pulls the extreme value back to a believable range. Capping is usually preferred for small datasets where every sample matters.

Example Code

```
df = make_dirty_iris()
df['sepal_length'] = df['sepal_length'].fillna(df['sepal_length'].median())
Q1, Q3 = df['sepal_length'].quantile([0.25, 0.75])
IQR = Q3 - Q1
lower, upper = Q1 - 1.5 * IQR, Q3 + 1.5 * IQR

# capping / winsorizing
df['sepal_length_capped'] = df['sepal_length'].clip(lower, upper)
print(df['sepal_length'].max(), df['sepal_length_capped'].max())

# removal
df_no_outliers = df[(df['sepal_length'] >= lower) & (df['sepal_length'] <= upper)]
print(df.shape, df_no_outliers.shape)
```

Exercise

1. Cap the outliers you found in `color_intensity` (previous exercise) using `.clip()` with the IQR bounds. Confirm the new maximum value is within bounds.
2. Instead, remove the outlier rows entirely and report how many rows remain.
3. In a short comment, explain which approach (cap or remove) you would choose for this dataset, and why.

1.7 Cleaning Messy Text & Categorical Values

Concept

Free-text and categorical columns are the most common source of “invisible” duplicates: "North", " north", and "NORTH" are three different strings to Pandas but the same category to a human. `.str.strip()` removes leading/trailing whitespace, `.str.lower()`/`.str.upper()` normalize case, and a manual `.map()` with a typo-correction dictionary fixes genuine misspellings that case/whitespace cleaning cannot.

Example Code

```
df = make_dirty_iris()
print(df['region'].unique())

cleaned = df['region'].str.strip().str.lower()
print(cleaned.unique()) # whitespace/case issues gone, typos remain

typo_map = {
    'north': 'North', 'nort': 'North',
    'south': 'South',
    'east': 'East', 'esat': 'East'
}
df['region_clean'] = cleaned.map(typo_map)
print(df['region_clean'].unique(), df['region_clean'].isnull().sum())
```

Exercise

1. Print `df['storage_region'].unique()` on `make_dirty_wine()` and identify every distinct spelling.
2. Clean it using `.str.strip().str.lower()`, then build your own typo-correction dictionary to map every variant to one of `'tuscany'`, `'piedmont'`, `'veneto'`.
3. Confirm with `.isnull().sum()` that your mapping covered every variant (zero NaN values after mapping). If you have leftover NaNs, inspect which spelling you missed.

2 Part II — Feature Engineering

From this point on we assume the data has already been **cleaned** using the Part I techniques (no missing values, no duplicates, no typos). Feature engineering reshapes *clean* data into a form models learn from more effectively. Copy the two helper functions below to your notebook – they return ready-to-use, already-clean versions of Iris and Wine, each with an added categorical column and an added date column so you have material to engineer with.

Example Code

```
def make_clean_iris_fe(seed=0):
    iris = load_iris()
    df = pd.DataFrame(iris.data,
                      columns=['sepal_length', 'sepal_width', 'petal_length', 'petal_width'])
    df['target'] = iris.target
    rng = np.random.default_rng(seed)
    sites = np.array(['North', 'South', 'East'])
    df['region'] = sites[rng.integers(0, 3, size=len(df))]
    base = pd.Timestamp('2023-01-01')
    offsets = rng.integers(0, 300, size=len(df))
    df['collection_date'] = pd.to_datetime(
        [base + pd.Timedelta(days=int(o)) for o in offsets])
    return df

def make_clean_wine_fe(seed=1):
    wine = load_wine()
    df = pd.DataFrame(wine.data, columns=wine.feature_names)
    df['target'] = wine.target
    rng = np.random.default_rng(seed)
    regions = np.array(['Tuscany', 'Piedmont', 'Veneto'])
    df['storage_region'] = regions[rng.integers(0, 3, size=len(df))]
    base = pd.Timestamp('2022-09-01')
    offsets = rng.integers(0, 120, size=len(df))
    df['bottling_date'] = pd.to_datetime(
        [base + pd.Timedelta(days=int(o)) for o in offsets])
    return df
```

2.1 Feature Scaling

Concept

Many algorithms (k-NN, SVMs, gradient descent-based models, PCA) are sensitive to feature *scale* – a column measured in thousands will dominate one measured in single digits, regardless of which is actually more important. **StandardScaler** rescales each column to mean 0, standard deviation 1. **MinMaxScaler** rescales to a fixed range (usually [0,1]). **RobustScaler** uses the median and IQR instead of mean/std, making it less sensitive to outliers you may have missed.

Example Code

```
df = make_clean_iris_fe()
num_cols = ['sepal_length', 'sepal_width', 'petal_length', 'petal_width']

from sklearn.preprocessing import StandardScaler, MinMaxScaler

scaler = StandardScaler()
```

```
scaled = scaler.fit_transform(df[num_cols])
print(scaled.mean(axis=0).round(2), scaled.std(axis=0).round(2))

mm = MinMaxScaler()
mm_scaled = mm.fit_transform(df[num_cols])
print(mm_scaled.min(axis=0), mm_scaled.max(axis=0))
```

Exercise

1. Apply `StandardScaler` to all 13 numeric features of `make_clean_wine_fe()`. Confirm the mean of every scaled column is approximately 0.
2. Apply `MinMaxScaler` instead, and confirm every scaled column now lies in `[0, 1]`.
3. Apply `RobustScaler` (from `sklearn.preprocessing import RobustScaler`) to the `proline` column alone and compare its output range to `StandardScaler`'s output for the same column. Which one is less affected by extreme values, and why?

2.2 Encoding Categorical Variables (scikit-learn)

Concept

Session 1 used `pd.get_dummies` for one-hot encoding. Inside an ML pipeline you will more often use scikit-learn's encoders directly: `OneHotEncoder` produces the same 0/1 columns but integrates with `Pipeline` and `ColumnTransformer`, while `OrdinalEncoder` assigns integer codes *in a category order you specify* – use it only when categories have a genuine rank (e.g. `small < medium < large`), never for nominal categories like a region name.

Example Code

```
df = make_clean_iris_fe()

from sklearn.preprocessing import OneHotEncoder, OrdinalEncoder

ohe = OneHotEncoder(sparse_output=False)
region_ohe = ohe.fit_transform(df[['region']])
print(region_ohe[:3])
print(ohe.get_feature_names_out())

sizes = pd.DataFrame({'size': ['small', 'large', 'medium', 'small']})
ordenc = OrdinalEncoder(categories=[['small', 'medium', 'large']])
print(ordenc.fit_transform(sizes[['size']]))
```

Exercise

1. One-hot encode `storage_region` from `make_clean_wine_fe()` using `OneHotEncoder`. Print the output shape and the generated feature names.
2. Create a small `quality` column with values `['low', 'high', 'medium', 'low', 'high']` and ordinal-encode it in the correct rank order `low < medium < high`.
3. In one sentence, explain what would go wrong if you used `OrdinalEncoder` (instead of `OneHotEncoder`) on `storage_region`.

2.3 Binning Continuous Variables

Concept

Binning converts a continuous column into discrete categories – useful for simplifying a noisy feature, building human-readable rules, or satisfying a model that handles categories better than raw numbers. `pd.cut` creates bins of **equal width** (you choose the boundaries), while `pd.qcut` creates bins of **equal frequency** (equal numbers of rows in each bin, with boundaries chosen automatically from the data's quantiles).

Example Code

```
df = make_clean_iris_fe()

df['petal_length_bin'] = pd.cut(
    df['petal_length'], bins=[0, 2, 5, 7],
    labels=['short', 'medium', 'long']
)
print(df['petal_length_bin'].value_counts())

df['sepal_width_quartile'] = pd.qcut(df['sepal_width'], q=4, labels=False)
print(df['sepal_width_quartile'].value_counts().sort_index())
```

Exercise

1. Bin `alcohol` from `make_clean_wine_fe()` into 3 equal-width categories 'low', 'medium', 'high' using `pd.cut` (choose sensible boundaries after checking `.describe()`).
2. Bin `color_intensity` into 4 equal-frequency groups using `pd.qcut`. Print how many rows fall in each group – are they roughly equal?
3. In a comment, explain one situation where `qcut` would be a *worse* choice than `cut`.

2.4 Extracting Features from Dates

Concept

A raw `datetime` column is rarely useful to a model directly, but the **parts** of it often are. Once a column has dtype `datetime64`, the `.dt` accessor unlocks `.year`, `.month`, `.day`, `.dayofweek` (Monday=0), and more. Combinations like `is_weekend` are simple derived flags that can be surprisingly predictive (e.g. weekend sales patterns, weekday sensor drift).

Example Code

```
df = make_clean_iris_fe()

df['collection_year'] = df['collection_date'].dt.year
df['collection_month'] = df['collection_date'].dt.month
df['collection_dow'] = df['collection_date'].dt.dayofweek
df['is_weekend'] = df['collection_dow'].isin([5, 6]).astype(int)
print(df[['collection_date', 'collection_month', 'collection_dow', 'is_weekend']].head(3))
```

Exercise

1. From `make_clean_wine_fe()`'s `bottling_date`, extract the month and the day-of-week into two new columns.
2. Create an `is_weekend` flag the same way as the example. What fraction of all bottling dates fall on a weekend?
3. Create a `days_since_start` column measuring how many days after '2022-09-01' each row's `bottling_date` is (hint: subtract two Timestamps, then use `.dt.days` on the result).

2.5 Transforming Skewed Features

Concept

Many models (especially linear ones) assume features are roughly symmetric. A strongly **right-skewed** feature (a long tail of large values, like income or price) can be pulled toward symmetry with `np.log1p` (log of $1 + x$, safe when $x = 0$) or `np.sqrt`. For a more principled, automatically-tuned transform, scikit-learn's `PowerTransformer` (Box-Cox/Yeo-Johnson) finds the best power transform for you. Always check `.skew()` before and after to confirm the transform actually helped.

Example Code

```
import numpy as np

rng = np.random.default_rng(0)
skewed = rng.exponential(scale=2.0, size=200) # a right-skewed example
print(pd.Series(skewed).skew()) # strongly positive skew

log_transformed = np.log1p(skewed)
print(pd.Series(log_transformed).skew()) # much closer to 0
```

Exercise

1. Check `.skew()` of every numeric column in `make_clean_wine_fe()`. Which column is the most skewed?
2. Apply `np.log1p` to that column and compare its skew before and after. Did it improve?
3. Use `sklearn.preprocessing.PowerTransformer` on the same column (`method='yeo-johnson'`, which handles zero/negative values unlike Box-Cox) and compare its resulting skew to your `log1p` attempt.

2.6 Interaction & Polynomial Features

Concept

Sometimes the relationship a model needs is not in any single column but in a **combination** of columns. A manual **interaction feature** (e.g. `petal_area = petal_length × petal_width`) encodes domain knowledge directly. `PolynomialFeatures` automates this: it generates every product and power up to a chosen degree, which is powerful but grows the feature count quickly – always check the resulting shape.

Example Code

```
df = make_clean_iris_fe()

df['petal_area'] = df['petal_length'] * df['petal_width'] # manual interaction

from sklearn.preprocessing import PolynomialFeatures

poly = PolynomialFeatures(degree=2, include_bias=False)
poly_feats = poly.fit_transform(df[['sepal_length', 'sepal_width']])
print(poly_feats.shape) # 2 columns → 5 (a, b, a^2, ab, b^2)
print(poly.get_feature_names_out(['sepal_length', 'sepal_width']))
```

Exercise

1. Create a manual interaction feature `phenol_density = flavanoids / total_phenols` on `make_clean_wine_fe()`.
2. Apply `PolynomialFeatures(degree=2)` to the `['alcohol', 'malic_acid']` columns. How many output columns are produced, and what are their names?
3. Repeat with `degree=3`. How many columns now? In one sentence, explain the risk of using a high polynomial degree on many input columns (hint: think about how fast the feature count grows, and overfitting).

2.7 Quick Feature Selection

Concept

Not every engineered feature is worth keeping. `VarianceThreshold` removes features that barely vary (and therefore carry little information). Checking `.corr()` against the target highlights features that are strongly related to what you are trying to predict – a fast, model-free first pass before deeper feature selection methods (covered in a later session).

Example Code

```
df = make_clean_iris_fe()
num_cols = ['sepal_length', 'sepal_width', 'petal_length', 'petal_width']

from sklearn.feature_selection import VarianceThreshold

vt = VarianceThreshold(threshold=0.2)
vt.fit(df[num_cols])
print(np.array(num_cols)[vt.get_support()]) # columns that survive

corr_with_target = df[num_cols + ['target']].corr()['target'].drop('target')
print(corr_with_target.abs().sort_values(ascending=False))
```

Exercise

1. Apply `VarianceThreshold(threshold=0.5)` to all 13 numeric features of `make_clean_wine_fe()`. Which features survive?
2. Compute each feature's absolute correlation with `target` and list the top 3 most correlated features.
3. Combine both checks: report any feature that is *both* low-variance **and** weakly correlated with

the target – these are your best candidates to drop first.

3 Capstone Exercise — Putting It All Together

Exercise

Mini-Project: End-to-End Cleaning & Feature Engineering on the Wine Dataset. Starting from `make_dirty_wine()`, write a single script that performs the following steps in order:

1. **Diagnose:** print `.info()`, missing-value counts, and duplicate-row count.
2. **Deduplicate:** remove exact duplicate rows.
3. **Fix types:** clean and convert `ash` to numeric, and `bottling_date` to datetime.
4. **Clean text:** standardize `storage_region` into exactly 3 canonical categories.
5. **Impute:** fill missing `magnesium` and `proline` values using an appropriate statistic.
6. **Treat outliers:** detect and cap outliers in `color_intensity` using the IQR method.
7. **Engineer features:** create at least one interaction feature, extract at least two date-based features from `bottling_date`, and bin one continuous column into 3 categories.
8. **Encode & scale:** one-hot encode `storage_region` and standardize all numeric features with `StandardScaler`.
9. **Select:** report the top 3 features most correlated with `target`, and any feature you would consider dropping due to low variance.
10. Print the final shape of your processed DataFrame and confirm `(.isnull().sum().sum() == 0)` that no missing values remain.

This mirrors the complete cleaning-to-features pipeline you will run at the start of nearly every project this semester, right before model training begins next session.

Appendix: Quick Reference Cheat Sheet

Data Cleaning

Function	Purpose
<code>.info()</code> , <code>.isnull().sum()</code>	Inspect dtypes, non-null & missing counts
<code>.duplicated()</code> , <code>.drop_duplicates()</code>	Flag / remove duplicate rows
<code>.str.strip()</code> , <code>pd.to_numeric()</code>	Clean whitespace; coerce text to numeric
<code>pd.to_datetime()</code>	Parse a text column into datetime
<code>.astype('category')</code>	Mark a column as categorical
<code>.fillna(stat)</code>	Impute with a constant or statistic
<code>groupby(col).transform()</code>	Group-wise imputation
<code>SimpleImputer</code>	Pipeline-friendly imputation (mean/median/mode)
IQR: $Q1 - 1.5 IQR$, $Q3 + 1.5 IQR$	Outlier detection bounds
Z-score: $(x - \mu)/\sigma$, $ z > 3$	Alternative outlier detection
<code>.clip(lower, upper)</code>	Cap (winsorize) outlier values
<code>.str.lower()</code> , <code>.str.strip()</code>	Normalize text casing/whitespace
<code>.map(dict)</code>	Correct typos / standardize categories

Feature Engineering

Function	Purpose
<code>StandardScaler</code>	Scale to mean 0, std 1
<code>MinMaxScaler</code>	Scale to a fixed range (e.g. [0, 1])
<code>RobustScaler</code>	Scale using median/IQR (outlier-resistant)
<code>OneHotEncoder</code>	One-hot encode nominal categories
<code>OrdinalEncoder</code>	Integer-encode <i>ordered</i> categories
<code>pd.cut()</code>	Equal-width binning
<code>pd.qcut()</code>	Equal-frequency binning
<code>.dt.year/.month/.day/.dayofweek</code>	Extract date-time components
<code>np.log1p()</code> , <code>np.sqrt()</code>	Reduce right-skew in a feature
<code>PowerTransformer</code>	Automatic Box-Cox / Yeo-Johnson transform
<code>col_a * col_b</code>	Manual interaction feature
<code>PolynomialFeatures</code>	Automatic polynomial & interaction terms
<code>VarianceThreshold</code>	Drop near-constant (low-information) features
<code>.corr()['target']</code>	Rank features by correlation with the target

End of Practical Session 2 Lab Sheet.